High-level approach for generating household data, dialogs, and extraction metrics

This page summarizes the end-to-end methodology across data generation, dialog generation, model training, Tinfoil-hardened hot-path construction, and control-overhead metrics.

Households	Dialogs	Extraction metrics	Models	Tinfoil-hardened hot-path	Control-overhead metrics
5,000	316	98.74%	3	0	98.74%
Synthetic household population	Multi-turn dialogs	Field-level extraction accuracy	Trained models	Tinfoil-hardened runtime components	Control-overhead reduction

Table of contents

- Overview
- Household data generation pipeline
- Dialog generation and dialog checks
- Matching, scoring, and errors
- Train holdout and pipeline
- Train and holdout test results
- Tinfoil-hardened hot path
- Control-overhead metrics

The benchmark is built as a controlled synthetic-data pipeline

Concepts households
Sample long-run household states from Census ACS PUMS data, then attach income, employment, utilities, public data sections, and discrete constraints.

Generate dialogs
Create long-context dialogs from structured household state, with explicit ground truth for what the assistant should extract from the full field-level schema.

Extract and score
Run structured extraction from dialogs, then score predicted versus ground truth with field-level accuracy, exact-match, and macro-F1 metrics.

Deployment scaffolds
Wrap the trained extractor in a Tinfoil-hardened runtime with attestation, encrypted inference, and control-overhead measurements.

Household data is scenario-driven and distributionally-aware

11 scenarios
Every household belongs to one named scenario. Those scenarios control income, employment, housing, utilities, and optional sections.

Hierarchical conditioning
83k-like population
The generated sample has a realistic size, hierarchical structure, and conditional dependencies rather than independent random fields.

Constraints
9 rule violations
The generated sample has no hard-constraint violations in the current artifact set.

Scenario coverage by 5,000 household synthetic population

Scenario	Count	Share
young_renter_no_college_paid_utilities	880	17.6%
affluent_couple_professionals_owned_utilities	765	15.3%
gig_independent_no_utilities	725	14.5%
high_earner_owner_one_has_partner	512	10.2%
single_mom_low_income	410	8.2%
multi_generational_extended_family	400	8.0%
financially_stressed_with_debt	385	8.00%
retired_couple_high_assets_no_income	380	7.60%
recently_graduated_paying_of_college	270	5.40%
divorced	245	4.90%
student	180	3.60%

Selected data generation diagnostics

Selected diagnostics for the household population and dialogs generated from the pipeline. Dialogs then illustrate how we check semantic coverage, marginal distributions, non-trivial consistency, dialog length, discourse effects, and unpaired cases.

Dialogs are generated in stages and checked for grounding and realism

Dialog generation has two main checks: grounding against the structured household state, and realism checks for unnatural phrasing, repetition, and over-regular turn-taking. Most optionality lives in the discourse layer, so the dialog can stay grounded while still sounding like a real conversation.

Grounded transcript construction
Dialogs are generated from the household-level profile and scenario context. It enforces transcript JSON structure, evidence fields, validation retries, and phone-to-text normalization.

Deterministic realism gate
A lexical-and-structure gate checks for unnatural phrasing, repetition, and over-regular turn-taking. Failed dialogs are rewritten with bounded retries.

Matching and scoring metrics

Metrics computed after record matching and grounded-aware field scoring

The evaluator compares extracted JSON to dialog-grounded ground truth, normalizes values, and reports field-level accuracy, exact-match rate, and error counts.

Record matching
Primary keys are used when available; otherwise the evaluator falls back to a stable profile hash.

Field scoring
Scalars, lists, and dictionaries are compared under explicit normalization rules, so formatting differences do not inflate or deflate the metrics.

Error types
Errors are split into missing, extracted, wrong, and unparsed cases, which makes it easier to separate model quality from formatting failures.

Scoring assumptions

Assumption	Implementation	May it matter?
Grounding-grounded evaluation	Where ground truth is_grounded=true, only fields present in grounded_fields enter the denominator.	Extraction is not penalized for facts never stated in the dialog.
JS exclusion	Primary keys and _id fields are excluded by default.	We evaluate semantic extraction, not synthetic ID generation.
Derived ratio fields	Fields ending in _ratio are excluded.	Ratio fields are derived and easier; excluding them keeps the main score cleaner.
Normalization assumptions	Phone numbers, booleans, and enums are normalized before comparison.	Helps avoid false errors from formatting differences, but assumes the normalizer is correct.

Train and holdout test

Train was used for pipeline development; holdout test is a held-out evaluation

The pipeline was developed on Tinfoil-hardened runtime components, so the train split is not a fully clean development number. The holdout split is the more honest generalization check for the deployed extraction stack.

Split summary

Split	Dialogs	Grounded GT records	Field-level records	Field-level accuracy	Exact-match rate	Errors
Train	216	2,118	12,711,125	98.74%	0.7407	Pipeline debugging and development
Holdout test	100	98	611,588	97.94 / 98.07	0.6735 / 0.6735	Held-out evaluation

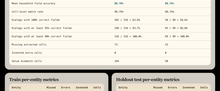
Tinfoil-hardened runtime

Train was used for pipeline development; holdout test is a held-out evaluation

The split Tinfoil-hardened runtime adds attestation, encrypted inference, gateway routing, and per-request measurement overhead. The application-level extraction metrics remain close to the non-private baseline, so the deployed stack keeps most of the extraction quality while adding strong runtime hardening.

Tinfoil-hardened run summary

Metric	Train (200)	Holdout test (98)
Mean household field accuracy	98.74%	98.74%
Exact-match match rate	98.74%	98.74%
Average with JSON correct fields	20 / 20 = 10.00	19 / 19 = 10.00
Average with at least one correct field	20 / 20 = 10.00	19 / 19 = 10.00
Average extracted fields	20 / 20 = 10.00	19 / 19 = 10.00
Average generated turns	5	5
Generated turns suffix	0	0
Total extracted rate	98	98



Control-overhead metrics

Control-overhead metrics show the cost of the Tinfoil-hardened path

Control-overhead metrics compare the Tinfoil-hardened path against a non-private baseline. The main question is whether attestation, encrypted inference, and gateway routing add enough latency or token overhead to make the deployed path impractical.

Pipeline

Household data generation pipeline

Each stage has its own checks before the next stage consumes its outputs

1. Synthetic data quality
Generate synthetic household populations from Census ACS PUMS data, then validate schema, distribution drift, cross-field logic, and rule violations before downstream use.

2. Dialog generation quality
Generate grounded multi-turn dialogs from structured household state, then check grounding, realism, and control-overhead behavior before training or deployment.

3. Model training and evaluation
Train extraction models on grounded dialogs, then evaluate on held-out splits with field-level accuracy, exact-match rate, and error breakdowns.

4. Tinfoil-hardened runtime
Wrap the trained extractor in a Tinfoil-hardened runtime with attestation, encrypted inference, and measured control-overhead impact.

Synthetic data quality

Synthetic data quality

Synthetic data quality checks validate schema, distribution drift, cross-field logic, and rule violations before downstream use.

Multi-scenario coverage
Household populations span 11 named scenarios with explicit coverage targets, so rare but important cases do not disappear into the average.

Hierarchical constraints
Income, employment, housing, utilities, and optional sections follow conditional dependencies rather than independent random fields.

Drift and validity checks
Distribution checks compare the generated sample against reference distributions, while rule checks catch impossible or inconsistent records before training.

Dialog generation

Dialog generation and checks

Dialogs are generated from structured household state, then checked for grounding, realism, and control-overhead behavior before they enter training or deployment.

Grounded transcript construction
Dialogs are generated from the household-level profile and scenario context. It enforces transcript JSON structure, evidence fields, validation retries, and phone-to-text normalization.

Deterministic realism gate
A lexical-and-structure gate checks for unnatural phrasing, repetition, and over-regular turn-taking. Failed dialogs are rewritten with bounded retries.

Control-overhead behavior
Control-overhead metrics compare the generated path against a non-private baseline, so the dialog layer can be checked for latency and token overhead before deployment.

Extraction quality

Extraction quality

Extraction quality is measured against grounded ground truth, with field-level accuracy, exact-match rate, and error breakdowns reported for train and holdout splits.

Grounding-aware evaluation
Where ground truth is_grounded=true, only fields present in grounded_fields enter the denominator, so extraction is not penalized for facts never stated in the dialog.

Field-level scoring
Scalars, lists, and dictionaries are compared under explicit normalization rules, so formatting differences do not inflate or deflate the metrics.

Error breakdowns
Errors are split into missing, extracted, wrong, and unparsed cases, which makes it easier to separate model quality from formatting failures.

Tinfoil-hardened runtime

Tinfoil-hardened runtime

The Tinfoil-hardened runtime adds attestation, encrypted inference, gateway routing, and measured control-overhead behavior around the extraction stack, so the deployed path can be checked for both quality and runtime cost.

Attestation and encrypted inference
Attestation is a prerequisite for trusting model outputs in a hardened runtime. The application-level extraction metrics remain close to the non-private baseline, so the main question is whether the hardened path adds enough latency and token overhead to remain practical.

Gateway routing and measured overhead
Control-overhead metrics compare the hardened path against a non-private baseline, so the runtime can be checked for latency and token overhead before deployment.

Control-overhead metrics

Control-overhead metrics

Control-overhead metrics show the cost of the Tinfoil-hardened path. The main question is whether attestation, encrypted inference, and gateway routing add enough latency or token overhead to make the deployed path impractical.

Latency overhead
Control-overhead latency compares the baseline and Tinfoil-hardened paths, so it shows how much the hardened path adds to end-to-end response time.

Token overhead
Control-overhead token overhead compares the two paths on token usage, so it shows whether the hardened path changes prompt or completion cost materially.

Model training

Model training and evaluation

Model training and evaluation turn grounded dialogs into structured extraction outputs, then compare predictions against held-out ground truth with field-level accuracy, exact-match rate, and error breakdowns.

Supervised extraction
Models are trained on grounded dialogs to map conversation text into structured household fields, so the task stays close to the deployed extraction interface.

Held-out evaluation
Train and holdout splits are reported separately, so the main comparison shows how the extractor behaves on unseen households rather than only on the training distribution.

Error breakdowns
Missing, extracted, wrong, and unparsed errors are reported alongside accuracy, so model quality can be separated from formatting and parsing failures.

Deployment

Deployment

Deployment wraps the trained extractor in a hardened runtime, so the deployed path inherits attestation, encrypted inference, and measured control-overhead behavior rather than only model quality.

Hardened runtime
Attestation and encrypted inference are the main runtime checks added on top of the extraction stack, so the deployed path can be checked for both quality and runtime cost.

Control-overhead monitoring
Control-overhead metrics compare the hardened path against a non-private baseline, so latency and token overhead can be tracked before and after deployment.

Next work

Research and product directions

1. Improve dialog realism with multi-model critique
Use a second-model critique step to reduce unnatural dialog artifacts, such as phone, telephone, SMS, or "reach out" when the transcript should stay in-person. This is the main realism lever after the current deterministic checks.

2. Compare structured output modes
Compare JSON-schema and tool-call extraction on the same models, prompts, and splits, so the deployed interface can be chosen on reliability and latency rather than only on prompt convenience.

3. Study dialog length versus extraction quality
Quantify how transcript length, turn count, and long-context drift affect field-level accuracy, so the system can choose when to split, summarize, or retrieve instead of extracting from one long dialog.

4. Add a prior-enhanced mode
Compare a rule-based extractor against the current model-based extractor, so the benchmark can separate easy structured recovery from the harder grounded extraction problem.

5. Build adversarial dialog attacks
Add a prompt-and-scenario suite for contradictions, indirect updates, and ambiguous phrasing, so the benchmark can stress-test the extractor beyond cleaner synthetic conversations.

6. Add a prior-enhanced mode
Compare a rule-based extractor against the current model-based extractor, so the benchmark can separate easy structured recovery from the harder grounded extraction problem.

structured generation / structured output constraints. Measure not only accuracy, but also parse failures, coercion issues, latency, retry rate, and cost.

extraction quality and latency. Identify the dialog size at which extraction should switch from single-pass processing to batching, chunking, or hierarchical extraction.

much information as possible from public or permissioned sources, such as public professional profiles or other available records. The dialog can then focus only on missing, ambiguous, or stale facts, shortening the conversation while still refreshing outdated information.

7. Make infrastructure failure-tolerant

Design graceful failover for API outages and provider degradation. For example, if GPT is unavailable, switch to Gemini or another provider, but only through a pipeline that has been explicitly tested and calibrated for that fallback model.

8. Evaluate multiple extractor models and ensembles

Test different LLMs as extractors and compare single-model, fallback, voting, and specialist-ensemble setups. Track accuracy, field-specific failure modes, latency, cost, and operational reliability.

9. Run real-world A/B validation

Compare this pipeline against the current production approach on real dialogs. Measure actual error patterns, human review burden, correction rates, and how extraction errors correlate with downstream product metrics.

ARTIFACT LINKS

Useful local references

These links point to files in this repository and should work when opening this HTML from the repo root.

Data generation

- 01 approach
- 01 README
- Scenario coverage CSV
- Scenario coverage plot

Dialogs

- 02 README
- DeepSeek prompt
- Train grounded GT
- Holdout test registry

Extraction and results

- 03 approach
- 03 README
- Train metrics
- Holdout test metrics
- Train discrepancy report
- Holdout test discrepancy report

Generated from local artifacts in this repository. Open from the repository root so relative image and document links resolve correctly.

Back to top